96. Параметры и аргументы функций

Параметры vs. Аргументы

Параметр функции (или ***«формальный параметр»***) — это переменная, создаваемая в объявлении функции:

```
void boo(int x); // объявление (прототип функции). x - это параметр

void boo(int x) // определение (также объявление). x - это параметр
{
}
```

Аргумент (или ***«фактический параметр»***) — это значение, которое передает в функцию вызывающий объект (caller):

```
boo(7); // 7 - это аргумент, который передается в параметр x
boo(y+1); // выражение y+1 - это аргумент, который передается в параметр x
```

Когда функция вызывается, все параметры функции создаются как переменные, а значения аргументов копируются в параметры. Например:

```
void boo(int x, int y)
{
}

boo(4, 5);
```

При вызове функции boo() с аргументами `4` и `5`, создаются параметры `x` и `y` функции boo() и им присваиваются соответствующие значения: `4` и `5`. Результатом будет `x = 4` и `y = 5`.

> ***Примечание***: В примере, приведенном выше, порядок обработки параметров в функции boo() будет справа налево, т.е. сначала создастся переменная `y` и ей присвоится значение `5`, а затем уже создастся переменная `x` и ей присвоится значение `4`. Порядок, в котором инициализируются параметры в круглых скобках функции, определяет каждый компилятор отдельно, так как C++ явно не указывает этот порядок обработки. С параметрами-переменными это не столь важно и критично, но если вы будете использовать в качестве параметров функции вызовы других функций (что является плохой практикой и не рекомендуется к использованию), то результат может быть неожиданным.

Рассмотрим следующую программу:

```
#include <iostream>

int prinX()
{
    std::cout << "x = 4\n";
    return 0;
}

int prinY()
{
    std::cout << "y = 5\n";
    return 0;
}

void prinAll(int a, int b) {}

int main() {

    prinAll(prinX(), prinY());  // в качестве параметров функции используются вызовы функций X() и Y()
    return 0;
}
```

Результат выполнения программы:

```
y = 5
x = 4
```

Хотя параметры не объявлены внутри блока функции, они имеют локальную область видимости. Это означает, что они создаются при вызове функции и уничтожаются, когда блок функции завершается:

```
void boo(int x, int y) // x и y создаются здесь
{
} // x и y уничтожаются здесь
```

Существует 3 основных способа передачи аргументов в функцию:

- передача по значению;
- передача по ссылке;
- передача по адресу.

Мы рассмотрим каждый из этих способов по порядку.